

我的 Codex 劇本寫作流程

從故事設定、版本管理到人機協作的實戰方法

我寫這份流程，主要是留給未來的自己。

萬一哪天年紀大了，忘記怎麼用 Codex 寫劇本，至少還有一套可以回頭照著做的工作方法。科技一直在變，ChatGPT、Codex、GitHub、雲端硬碟未來都可能換介面、換名稱、換規則，但創作流程本身可以持續修正、沿用。

這不是唯一正確的方法，也不是要大家照表操課。它只是我目前實測後，覺得比較能兼顧效率、品質、版本管理與創作安全的一套做法。

整體來說，我把工作分成四個層次：

1. ChatGPT 負責前期開發：想故事、整理設定、建立結構。
2. Codex 負責文本執行：依規格寫場次、修稿、整理文件。
3. 其他 AI 負責評析：從不同角度挑錯、找問題、提出修改方案。
4. GitHub、Dropbox 與離線備份負責保存：讓每一次修改都有跡可循。

人類仍然是最後的編劇、導演與製作人。AI 可以幫忙想、幫忙寫、幫忙挑錯，但留下什麼、刪掉什麼、故事最後要說什麼，還是由你決定。

第一部分：前置建置流程

步驟一：建立本機專案資料夾

先決定作品名稱，例如：

電影劇本《0000》

接著在電腦的 Dropbox 或主要工作硬碟內建立專案主資料夾，例如：

0000/

我建議把「工作區」與「封存區」分開，不要全部混在一起。工作區是每天會修改的地方；封存區則是手動複製進去、原則上不再自動同步更新的版本。

可以採用以下概念：

```
0000/  
├── 01_Workspace/  
├── 02_Research/  
├── 03_Archive/  
│   ├── Canon/  
│   ├── Codex/  
│   └── Conversation/  
└── 04_Exports/
```

Dropbox 可以作為平常同步工作區，但不要把它當成唯一備份。同步很方便，也代表錯誤刪除、覆蓋或格式跑掉時，可能一起同步出去。

真正重要的版本，仍要定期手動複製到另一顆硬碟、NAS 或不會自動同步的異地資料夾。

手動備份雖然麻煩，但它的價值在於：你必須親自確認「這一版真的值得保存」。

步驟二：建立 GitHub 私人版本庫

接著到 GitHub 建立新的 Repository。

命名原則不必太死板，但建議使用英文片名加作品類型，例如：

```
0000-Screenplay  
0000-Teleplay  
0000-Stageplay
```

電影劇本可以用 Screenplay，電視劇或影集可以用 Teleplay，舞台劇可以用 Stageplay。重點不是名稱多漂亮，而是未來一看就知道是哪一部作品。

Repository 說明可以簡單寫：

```
電影劇本《0000》
```

或：

```
Feature screenplay project: 0000
```

最重要的是：劇本專案請設為 **Private**。

公開的 Repository 可以拿來放流程範本、工具、空白模板或教學文件；真正還在開發、準備投稿、尚未發表的劇本，最好另外建立私人版本庫，不要和公開範本混在一起。

GitHub 在這裡的功能，不是單純「放檔案」，而是讓你保留每一次修改的差異、時間與版本紀錄。日後你要回頭找某一版人物設定、某一場戲、某一句對白從哪裡被改掉，Git 紀錄會比單靠一堆「最終版、最終版 2、真正最終版」可靠得多。

步驟三：讓本機工作區與 GitHub 對齊

這一步的觀念要先釐清：

Codex 主要是在你的本機專案資料夾或 worktree 中工作；GitHub 則是遠端版本庫。

所以流程不是把 GitHub 網址丟給 Codex 就結束，而是：

5. 在本機建立或 clone 專案資料夾。
6. 確認該資料夾已連接到 GitHub Repository。
7. 用 Codex 開啟這個本機專案資料夾。
8. 每次工作完成後，檢查修改內容、commit，再 push 到 GitHub。

Codex 對話名稱也建議統一，例如：

電影劇本《0000》

未來同時開多個專案時，才不會搞混。

每天開始工作前，先確認目前是哪一部作品、哪一個版本、哪一份 Canon、哪一個場次。這種看似瑣碎的確證，其實能避免 AI 在錯的故事裡努力半天。

步驟四：先做故事研究，不要直接叫 AI 亂編

如果是完全原創作品，可以先和 ChatGPT 討論：

- 主題
- 類型

- 觀眾定位
- Logline
- 故事大綱
- 人物設定
- 世界觀
- 結構節拍
- 商業定位
- 預算尺度
- 影像風格

如果是改編真實故事、歷史事件、人物傳記、案件、職業或專業領域，先做研究。

AI 很適合幫你快速建立「問題地圖」，例如告訴你這個人物需要查哪些年代、職業規則、事件順序、地理位置、組織架構或法律背景。

但不要把 AI 當成唯一資料庫。

尤其是真實人物、歷史事件、航空、醫療、軍警、法律、金融、政治、宗教、犯罪、戰爭等題材，最好建立一份 Research Log，記錄：

資料名稱：
來源：
日期：
關鍵內容：
可用於哪一場戲：
尚待查證事項：

AI 可以幫你整理資料、找問題、提出推測，但關鍵事實還是要回到可靠來源確認。否則你可能寫出一個很流暢、很像真的、其實根本錯誤的劇本。

第二部分：從故事發想到 Canon

步驟五：先從小到大建立故事

劇本創作的基本順序，和傳統編劇理論其實沒有差太多。

先決定：

9. 主題：這個故事真正想談什麼？
10. Logline：主角是誰、目標是什麼、阻力是什麼、代價是什麼？
11. 類型：愛情、災難、動作、驚悚、家庭、歷史、科幻、黑色喜劇？
12. 長度：短片、長片、電視電影、影集、舞台劇、音樂劇？
13. 結構：三幕劇、Save the Cat、五幕劇、分集節拍或其他形式？
14. 人物：誰是主角、誰是對手、誰推動故事、誰承擔情感？
15. 故事大綱：事件如何發生、轉折在哪裡、結尾如何落下？

一開始不必就逼自己寫到五千字。

先寫一千字故事大綱，確認方向對不對；再慢慢擴寫成三千到五千字的完整故事梗概。人物小傳、角色秘密、童年經驗、人物關係、角色弧線，也可以和大綱同步發展。

這個階段最適合反覆討論。

你可以問 ChatGPT，也可以問其他 AI，甚至可以故意找不同模型挑戰你的設定。故事不是第一版就定型，而是在反覆推敲中逐漸長出骨頭。

步驟六：建立 Canon 與 Narrative Spec

我習慣把劇本設定檔叫作 Canon。

Canon 不是神聖不可改的聖經，而是「目前這一版故事不可隨便違背的規則」。

一份好的 Canon 至少包括：

- 故事核心主題
- 世界觀與時空背景
- 人物年齡、職業、經歷與關係
- 人物動機、秘密、弱點與底線
- 劇情時間線
- 已經發生、不能推翻的重要事件

- 不可違背的設定
- 語氣、風格與禁忌
- 專有名詞、地名、組織名稱、專業術語
- 尚未決定、仍可調整的問題

但 Canon 比較像設定資料庫。

真正交給 Codex 執行時，還要有一份「當前任務規格」，也就是 Narrative Spec。

Narrative Spec 要講得更具體，例如：

這次要寫第幾集、第幾場。
這一場的戲劇目的。
角色各自想得到什麼。
衝突如何升高。
這一場結束後，觀眾應該知道什麼。
禁止改動哪些既有設定。
字數、頁數、劇本格式與語氣要求。

簡單來說：

Canon 是世界規則。
Narrative Spec 是本次施工圖。

不要每次都叫 Codex 「照設定寫一場戲」。

那種指令太模糊，它很容易自己補空白、自己加新設定、自己替你決定角色的命運。你想要的是助手，不是讓它趁你不注意時接管編劇室。

步驟七：建立 3C 封存系統

我習慣把封存資料分成三個主要類別，簡稱 3C：

Canon
Codex
Conversation

Canon

Canon 資料夾存放所有故事設定、人物表、時間線、版本規格、Narrative Spec 與研究摘要。

最推薦的主格式是 Markdown，也就是 .md 檔。

Markdown 的好處是：

- 人可以直接閱讀。
- AI 容易理解。
- 不會被 Word 格式綁死。
- 可以放進 Git 做版本追蹤。
- 未來要轉成 Word、PDF、網頁或其他格式都方便。

檔名建議統一，例如：

```
0000_Canon_20260622_v001.md
0000_Narrative-Spec_20260622_v001.md
0000_Character-Bible_20260622_v002.md
```

不要再用「最新版」、「最後版本」、「最後版本真的最後」這種命名法。

版本一定會再改。先承認這件事，檔案才不會失控。

Codex

Codex 資料夾用來保存每天的產出。

建議每個工作日建立一個日期資料夾，例如：

```
20260622/
```

裡面可以放：

```
EP01_S01-S03_Draft.md
EP01_S01-S03_Draft.docx
EP01_S01-S03_Draft.pdf
Revision-Notes.md
Change-Log.md
```

不過有一個原則很重要：

Markdown 或純文字劇本應該是主要母檔；Word 與 PDF 是輸出格式。

因為 Word 與 PDF 適合閱讀、列印、投稿與交換，但不適合追蹤細微差異。真正要做版本管理的，最好還是以 Markdown、純文字或其他可比較的文字檔為主。

Conversation

Conversation 資料夾保存每次重要作業的對話紀錄、工作指令、修改理由與決策摘要。

可以是 .txt、.md 或匯出的對話檔。

它的用途不是保證未來一定能打贏任何著作權爭議，而是保留你的創作軌跡：

- 你何時提出概念。
- 你如何修改角色。
- 你如何取捨不同版本。
- 哪些內容是你的人類決策。
- 哪些內容是 AI 提出、再由你改寫或採用。

除了對話紀錄，我建議再加一份簡單的 Decision Log：

```
日期：
問題：
可選方案：
最後決定：
決定理由：
影響場次：
```

這會比單純保存一大串聊天紀錄更有用。

第三部分：進入 Codex 實際寫作

步驟八：建立專案工作架構

當故事架構大致成熟後，再請 Codex 建立寫作專案。

你可以參照自己的 narrative-pipeline 流程範本，把每一部作品拆成：

```
00_brief
01_development
02_spec
03_outline
04_scenes
05_critiques
06_revisions
07_exports
```

這個結構的好處是：故事設定、場次草稿、評析報告、修訂版本與最後輸出不會全部混在一起。

不要把所有東西都丟進同一個資料夾，然後期待未來的自己還記得哪一份才是真的。

你現在也許記得，但半年後、三年後、十年後，連你自己都可能看不懂。

步驟九：建立風格參照，不要幻想 AI 真的複製了你

把自己過去寫過的劇本、小說、短篇、舞台劇或對白本放進 **Style** 或 **references** 資料夾，讓 **Codex** 作為風格參照。

但我建議不要再叫它「訓練人格」或「完全學會你的風格」。

比較準確的說法是：

你提供作品樣本，讓它分析並參照你的句型、節奏、場景描述、對白長度、人物互動、敘事傾向與常用語氣。

接著，請它整理成一份 **Style Guide**，例如：

句子長度偏好：
場景描述密度：
對白節奏：
角色說話習慣：
避免使用的詞彙：
避免出現的敘事毛病：
是否使用鏡頭語言：
是否允許旁白：
是否偏好留白：

只要是自己的作品，或你已取得授權的文本，都可以作為參照資料。

流程可以被別人學走，工具也可以被別人複製，真正的美學選擇、人生經驗、情感記憶、價值判斷與對世界的看法，並不會因為丟幾份檔案進去就被完整蒸餾。

AI 可以模仿表面特徵，但它不知道你為什麼會在某一個場景裡，讓一個角色沉默三秒。

那個選擇還是你自己的。

坦白說，我實測過後也發現：**Codex** 有時候確實寫得比我更簡潔、更俐落、更像一個沒有情緒包袱的職業寫手；但它模擬我的版本，往往還是差強人意。

所以風格參照可以做，但不要把它當成複製靈魂的技術。

步驟十：先寫前十頁測試

不要一開始就叫 Codex 把整部劇本寫完。

先寫前十頁。

好萊塢劇本閱讀習慣裡，前十頁常常是讀者決定要不要繼續讀下去的試金石。它未必需要把所有事情交代清楚，但至少要让別人知道：

- 這是什麼類型的故事。
- 主角是誰。
- 主角的問題是什麼。
- 這個世界有什麼吸引力。
- 觀眾為什麼要繼續看下去。

如果你準備投稿優良劇本、優良劇集、影展、補助案或其他比賽，先把格式範本、投稿規則、頁數限制、字體規格與禁忌餵給 Codex。

不要等到劇本寫完才發現格式不符合規定，然後又花一堆時間整本重排。

前十頁測試時，我建議一次要求三份輸出：

1. 劇本草稿
2. 本場戲的意圖說明
3. 與 Canon 不一致或仍待確認的問題清單

這樣你不只看成果，也看得出它到底有沒有理解你的故事。

步驟十一：讓 Codex 提出寫作計畫，再分批寫作

確認前十頁方向正確後，再請 Codex 提出寫作計畫。

不要一口氣叫它寫完整部劇本，除非你真的趕死線，而且能接受後續要大修。

一次叫 AI 寫太長，通常會出現幾個問題：

- 後段節奏開始鬆掉。
- 角色逐漸失去原本的聲音。

- 前面埋的伏筆被忘掉。
- 場景變得重複。
- 內容看似很多，其實沒有推進。
- 修稿成本比拆開寫更高。

我認為比較合理的做法，是一次推進十到二十頁，或者以三到五場戲為一個工作單位。

一部約一百二十頁的電影劇本，純生成速度理論上可以在六到十二個工作天內跑完；但把審稿、修訂、版本整理、重新評析都算進去，抓兩週左右比較務實。

AI 可以很快，但你不必跟著它一起失控。

真正重要的不是「今天生了幾頁」，而是「今天留下來的幾頁，還值不值得明天繼續寫」。

步驟十二：多模型評析，但不要把創作交給投票

每一批草稿完成後，可以交給不同 AI 評析。

例如：

- ChatGPT：故事結構、角色弧線、主題與情感。
- Claude：長文閱讀、邏輯一致性、節奏與人物動機。
- Gemini：資訊整理、研究比對、場景問題。
- Grok 或其他模型：提供不同視角、找出你慣性忽略的地方。

但不要只是問：

這個故事好不好？

這種問題通常只會換來一堆空泛稱讚。

應該問得更具體，例如：

請找出本集最弱的三個場景。
請指出角色動機不足的地方。
請檢查是否違反 **Canon**。
請分析節奏在哪一頁開始鬆掉。
請找出不必要的說明性對白。
請提出三種不同強度的修改方案。
請分別從觀眾、製作人與劇本競賽評審角度評析。

我把這個流程叫作「手動對抗式生成」。

不是讓不同 AI 互相打架，而是讓不同模型替你找盲點，再由你決定哪些意見值得採納。

最後永遠要回到人類決策。

AI 可以提出一百個意見，但你不需要全部改。劇本不是民主投票，也不是誰的模型分數高就一定比較好。

步驟十三：新點子先更新 Canon，再修改劇本

寫到一半想到新點子很正常。

問題是，很多人一想到新點子，就直接叫 AI 大改整本劇本。結果改完才發現：人物設定、時間線、前後伏筆、角色關係全部被扯爛。

比較好的流程是：

```
新點子
→ 討論可行性
→ 更新 Canon
→ 更新 Narrative Spec
→ 指定受影響場次
→ 請 Codex 進行局部修訂
→ 產出 Change Log
→ 人工確認
```

這樣做不只是省使用額度，更重要的是避免整個故事系統失控。

ChatGPT 比較適合發想、討論、整理與提出不同可能性；Codex 比較適合按照明確規格執行、整理檔案、改寫場次與產出版本。

一個偏編劇室討論，一個偏執行製作。

把工作分開，效率通常會比叫同一個工具包辦所有事情更好。

第四部分：每日收工與版本保存

步驟十四：每天工作結束前的固定流程

每天收工前，我建議固定完成以下事項：

1. 閱讀當天產出。
2. 確認哪些內容正式採用。
3. 更新 Canon 或 Narrative Spec。
4. 寫入 Change Log 與 Decision Log。
5. 檢查檔案差異。
6. Commit 當日版本。
7. Push 到 GitHub。
8. 將當日重要輸出手動複製到異地封存資料夾。
9. 保存重要對話紀錄。
10. 最後才讓 Dropbox 完成同步。

Commit 訊息也不要亂寫。

例如：

```
feat(ep01): 完成第一集 S01-S03 初稿
revise(canon): 更新鄧子琪空難後創傷設定
fix(ep02): 修正角色年齡與時間線衝突
critique: 新增 Gemini 與 Claude 評析摘要
```

未來你要回頭找「哪一天改掉父親角色的設定」、「哪一版把結局改成雙主角」、「哪一次刪除某場戲」，都會容易很多。

如果使用自動化寫作，更要避免讓它直接改你正在工作的主資料夾。

最好讓自動化任務在獨立工作區或 worktree 中產出，等你看過、修改過、確認沒問題，再合併回正式版本。

自動化可以幫你做重複工作，但不應該繞過你的審核。

第五部分：初稿完成後，回到紙本

步驟十五：把初稿印出來看

初稿完成後，記得輸出 PDF。

我自己的省錢做法是：用家用黑白雷射印表機，每三十頁雙面列印，再用雙孔打洞器與塑膠束帶固定。

這種方式不漂亮，但便宜、牢靠、好翻，也方便手寫。

我不太喜歡雙腳釘裝訂，尤其劇本一厚就很麻煩。電視劇或長篇影集動輒五百頁以上，雙腳釘容易鬆脫、散頁，而且翻閱不順。

當然，已經在業界工作、需要交製作單位、經紀人、劇組或正式評審的人，還是要依照對方規格處理。

但對我們這種還在開發、還在反覆修改的創作者來說，方便閱讀比形式更重要。

我習慣在紙本第一頁手寫日期與版本，例如：

2026.06.22
第二稿

這不只是儀式感。

紙本閱讀會讓你看到螢幕上看不到的問題：段落太長、場景太像、對白太滿、翻頁節奏不對、人物講話都長得一樣。

AI 可以幫你加速寫作，但紙本還是能逼你慢下來看作品。

第六部分：發表、投稿與賣劇本

如果有經紀人、製作人、公司或合作夥伴願意接手開發，當然最好。

如果沒有，也可以考慮：

- 參加優良劇本、優良劇集或相關競賽。

- 放在個人網站、劇本部落格或作品集。
- 改寫成小說、漫畫、舞台劇、音樂劇或其他 IP 形式。
- 製作企劃書、角色介紹、提案簡報與試讀本。
- 尋找影像創作者、導演、製片或小型工作室合作。

在台灣，參加優良劇本或優良劇集競賽，仍然是讓作品進入業界視野的一條路。

但比賽規則永遠優先於你的個人工作流程。

字數、頁數、匿名規定、格式、檔名、封面、是否可公開發表、是否允許 AI 協作、是否需要授權聲明，全部都要逐項確認。

如果作品改編自真實人物、新聞事件、既有小說、歷史資料或他人作品，也要額外評估資料來源、改編授權、隱私、名譽與其他可能風險。

不要因為 AI 寫得快，就以為作品可以跳過這些現實問題。

結語：劇本永遠是改出來的

最後還是那句老話：

劇本是改出來的。

不要太糾結第一稿是不是曠世傑作，不必一開始就逼自己寫成莎士比亞再世。

在作品還沒有拍成影視、沒有被觀眾看見、沒有真正進入產業流程之前，最重要的事情只有一件：把它寫完。

寫完之後再改。

改完之後再讓別人看。

就算作品真的賣出去，未來也可能被導演、製作人、演員、平台、預算與拍攝條件改得面目全非。

這不是創作失敗，而是影視工業的現實。

你真正能完全掌握的，只有寫作當下那個自由的世界。

AI 可以替你整理資料、生成場景、修改格式、追蹤版本、分析問題，甚至偶爾寫出比你更俐落的對白。

但那個故事為什麼存在、角色為什麼必須受傷、結局為什麼要留下那一句話，最後還是只有你知道。

流程可以複製。

工具可以共享。

真正無法被複製的，是你決定讓什麼故事活下去。